MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión E del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_crisostomo_israel
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/crisostomo_israel/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código

¿Qué significa que un objeto sea una instancia de una clase? Explica si dos instancias deben tener iguales valores en sus atributos.

Tu respuesta

```
si un objeto sea una intancia nos da a entender que este se puede usar vrias clases, las
intancias pueden tener valores iguales ya que estan en lugares diferentes de la memoria:
ejemplo si una clase se compara con los planos de una casa, en esta parte se defines las
ventanas, puertas, habitaciones etc, mientras que el objeto se compararia con la casa como
tal, al depender de una sola clase se puden crear varios objetos(casa), aunque sean iguales no
son identicas, ya que cada uno esta en un espacio de memoria difentes y si los atributos ya
que son el nombre y dierccion no pueden ser iguales ya que cada uno es diferente
```

Corrección. La analogía del plano (clase) y la casa (objeto) capta bien que una instancia es un objeto concreto creado a partir de la clase y que de una clase salen varios objetos. La respuesta se contradice: primero dice que las instancias pueden tener valores iguales y al final que sus atributos no pueden ser iguales. Lo correcto es que comparten la definición de atributos y métodos, pero cada una guarda sus propios valores, que pueden coincidir o no.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué diferencia hay entre un parámetro y un argumento de un método? Ilústrala con una llamada sencilla.

Tu respuesta

```
un parametro es lo que vivie dentro de una clase y le da sus caracteristicas , el argumento es el que le da nombre  y solo se ejecuta una ves mientras se esta ejecutando
```

Corrección. Es incorrecta: un parámetro no es algo que vive en la clase y le da características, ni el argumento es lo que le da nombre. El parámetro es la variable declarada en la cabecera del método (int cantidad en cargar(int cantidad)) y el argumento es el valor que se pasa al llamarlo (el 3 en cargar(3)).

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué es el ámbito de una variable local? Da un ejemplo de dónde se puede usar y dónde deja de estar disponible.

Tu respuesta

```
 una variable local a difencia de una variable de intancia, se caracterisa porque nace y nuere dentro del programa, un ejemplo seria si tenemos un objeto (perro)  y  clase (tobi), la variable local la podemos sacar de string (ladrido) , ya que mintras esta se este ejecutando la variable local(ladrido) se creea, una vez que termine esta se muere :(  ya que solo vive dentro de ladrido
```

Corrección. Conserva la idea válida de que la variable local se crea mientras se ejecuta su parte del código y deja de existir al terminar. Pero dice que vive en todo el programa, el ejemplo de perro y tobi confunde objeto, clase y variable, y no muestra el límite de un bloque. El ámbito es la región del código donde se puede usar el nombre: una local declarada dentro de las llaves de un if o de un método solo se usa ahí y fuera de ellas ya no existe.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué guarda una variable de tipo arreglo? Explica qué ocurre al asignarla a otra variable del mismo tipo.

Tu respuesta

```
la variable de tipo arreglo guarda datos de tipo int o boolean por ejemplo, cuando asignamos un valor a una del mismo tipo esta
```

Corrección. Conservas una idea válida: el arreglo agrupa datos de un tipo, como int o boolean. Pero no dices que la variable guarda una referencia al arreglo, y la explicación de la asignación quedó sin terminar. Al asignarla a otra variable se copia esa referencia: ambas apuntan al mismo arreglo (son alias, no hay copia), así que un cambio hecho por una se ve por la otra.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Qué hace break dentro de un case de un switch clásico? Explica qué puede ocurrir si falta.

Tu respuesta

```
el break es la puerta de "SALIDA", es el que le diace al codigo cuando salir, y en el ejemplo de un switch un ejemplo seria un pasillo lleno de puertas el switch sin una intruccion empezara a abrir todas las puertas, si le decimos al programa que sea "2" esta se abrira, y despues pasara al break, y termina el programa, en caso de que falte en el codigo este se empezara compilar una y otra vez hasta agotar la memoria, o que un factor exerno lo detenga,
```

Corrección. Aciertas en que break es la salida del switch. El error está en decir que, si falta, el código se vuelve a compilar o a ejecutar sin fin hasta agotar la memoria. Lo que pasa es que la ejecución sigue por los case siguientes sin volver a comparar sus etiquetas, hasta encontrar un break, un return o el final del switch.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
Integer original = 7;
Integer copia = original;
int vuelta = 0;
do {
    original = original + 2;
    System.out.println(original / 2);
    vuelta++;
} while (vuelta < 3);
String texto = "E" + copia + 1;
System.out.println(texto);
System.out.println((double) (original / 2));
```

Tu respuesta

```
4
E8
4.0
```

Corrección. La primera línea, 4, es correcta. Faltan 5 y 6, porque el do-while da tres vueltas (original vale 9, 11 y 13). "E8" no es correcto: "E" + copia + 1 concatena de izquierda a derecha y da E71, no suma 7+1. En "4.0" el formato .0 del cast a double es correcto, pero el valor es 13/2 = 6, así que la línea correcta es 6.0.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String codigo = null;
boolean igual = codigo.equals("R3");
System.out.println(igual);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
no compila por el null que es vacio, este no se puede ejecutar ya que no hay nada en memoria
```

Corrección. La clasificación es incorrecta: el código sí compila y truena al ejecutarse. Sí ve que el null es la causa y que no hay objeto en memoria, aunque no dice que el problema es llamar equals sobre una referencia null, que lanza NullPointerException.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
double voltaje = 7.6;
int lectura = voltaje;
System.out.println(lectura);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
en este si compila, su salida va ser 7.6 ya que el double votaje nos da el valor, al usar int
lectura esta va a leer el votaje y nos dara su valor y la salida es igual al votaje que es 7.6
```

Corrección. Es incorrecta: no compila, porque Java no convierte implícitamente un double en int, ya que se perdería precisión. Haría falta un cast explícito como (int) voltaje, y aun así el resultado sería 7, no 7.6.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
Integer reserva = 4;
int nueva = reserva + 3;
System.out.println(nueva);
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
esta el interger de reserva nos da 4, esta palabra reservada "envuelve" al valor de reserva ,
al no tener el valor de reserva al momento de compilar no tendra la dirreccion de memoria y
truena
```

Corrección. Es incorrecta: el código funciona e imprime 7. Reconoce que Integer envuelve el valor, pero Java lo desempaqueta automáticamente (unboxing) para sumar 4 + 3. No hay ningún fallo de memoria.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada banda nace vacía con capacidad propia fija. El límite de seguridad común es 12 piezas. agregarPieza() suma una y consultarPiezas() responde cuántas hay. Supón que solo se llama agregarPieza cuando ambos límites lo permiten.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Cuál es final de instancia y cuál static final: capacidad o límite de seguridad?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
static int agregarPieza(int consultarPiezas)
```

Justificaciones

```
//por falta de timpo no pude terminar
```

Corrección. Solo escribió una cabecera, static int agregarPieza(int consultarPiezas), sin cuerpo, y además mezcla los dos métodos y no deberían ser static. Faltan los atributos, la constante, el constructor, los métodos completos, el reto y las justificaciones. Modelo esperado: final int capacidad (de instancia), int piezas (de instancia y mutable), static final int LIMITE_SEGURIDAD = 12, un constructor con this.capacidad = capacidad, agregarPieza() { piezas++; } e int consultarPiezas() { return piezas; }.

R4 · Código con repetición

Pregunta

Un equipo parte con carga unidades. En cada paso suma aporte. Devuelve cuántos pasos necesita para alcanzar o superar meta usando while. Si ya llegó, devuelve 0. Carga y meta están entre 0 y 10000; aporte está entre 1 y 100. No uses una fórmula de división.

```
static int pasosCarga(int carga, int aporte, int meta)
```

Ejemplos (argumentos → retorno):

5, 4, 16 → 3 · 16, 4, 16 → 0 · 20, 4, 16 → 0

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
static int pasosCarga(int carga, int aporte, int meta)

int pasosCarga = 0;
int aporte aporte++;
int meta = 0<=10000;
} while ( aporte < 1 ++ ,i);
System.out.println();
System.out.println(();

//por falta de timpo no pude terminar
```

Corrección. La cabecera es correcta. No hay un while reconocible, el contador pasos nunca aumenta, carga no acumula aporte (en su lugar hace aporte++), no hay return y el método imprime, lo que el enunciado prohíbe. La solución esperada es: int pasos = 0; while (carga < meta) { carga += aporte; pasos++; } return pasos;

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Decide si un banco de pruebas puede arrancar. Recibes el porcentaje de batería y si la tapa está cerrada. Aplica la primera regla que corresponda, en el orden de la tabla. Usa if / else-if / else, con && y ||. No imprimas ni leas del teclado.

```
static String permiso(int bateria, boolean tapaCerrada)
```

Regla o valor recibido	Texto devuelto
Batería menor que 0 o mayor que 100	"INVALIDA"
Tapa cerrada y batería al menos 60	"NORMAL"
Tapa cerrada y batería al menos 20	"AHORRO"
Cualquier otro caso	"BLOQUEADO"

Tu respuesta

```
static String permiso(int bateria, boolean tapaCerrada);

for (int bateria = 0; i < 100; i ++ {
    if (0 > 100)
          (System.out.println("INVALIDA");{
        } else
   if (tapaCerrada && bateria ≤ 60);
      (System.out.println("INVALIDA");{
```

```
        } else
      if (tapaCerrada && bateria ≤ 20);
      (System.out.println("AHORRO") {
       }else
(System.out.println("BLOQUEADO");
return 0;
```

Corrección. La cabecera es correcta. Usas una cadena if/else-if/else en el orden de la tabla y dejas el resto en BLOQUEADO, así que la tapa abierta bloquea. Errores: el for sobra; "0 > 100" no revisa bateria (lo correcto es bateria < 0 || bateria > 100); las comparaciones van al revés (deben ser tapaCerrada && bateria >= 60 y tapaCerrada && bateria >= 20); la primera rama devuelve INVALIDA en vez de NORMAL; y el método imprime y hace return 0 en lugar de devolver el String.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.